

2.1 AVR Microcontroller – Multiplexed 7-Segment Countdown Display

OBJECTIVE AND OVERVIEW

In this task, we had to design a timer device that counts from 30 down to 00 using an ATmega328p AVR microcontroller and two 7-segment displays. The design and simulation was done in Simulide and the code for the same was written in Microchip Studio in C. After the build of the code, a hex file was obtained in the Debug directory and was loaded onto the microcontroller in Simulide. The simulation was run, leading to successful completion of the task with desired results as expected.

COMPONENTS USED

- ATmega328p microcontroller
- Two 7-segment (Common cathode) displays
- Two NOT gates
- 7 current limiting resistors (220Ω)
- Connecting wires

THEORY AND CONCEPTS

The microcontroller is a device used to upload a set of instructions (code) according to which it then carries out the processes in the circuit automatically. The microcontroller has many ports, out of which ports D0 to D6, B0 and B1 are of interest to us. We used a combined, or rather an internally connected 7-segment display by utilizing the “properties” of the seven segment display by right-clicking on it in Simulide. Had we not used this option, we would have ports of C series as interest to us as well.

The ports D0-D6 on the microcontroller, which are 7 in number, correspond to each of the 7 segments or bars of the display, labeled a, b, c, d, e, f and g. When the port goes HIGH, it lights up the corresponding segment it was connected to on the display. The ports B0 and B1 are used to decide which part of the multiplexed two 7-segment display gets ‘activated’ or displayed. We connected port B0 to the left one and port B1 to the right one according to our design. According to theory, in a common cathode 7-segment display, the digit selection i.e. to decide which part glows, that pin must go LOW. The reason is straightforward – each of the segments (a to g) are anodes and the common pin is the one which decides if that display glows or not; so for current to flow in the segment to light up the LED, it must flow from the positive side (anode) to the negative side (cathode, the common pin which is set

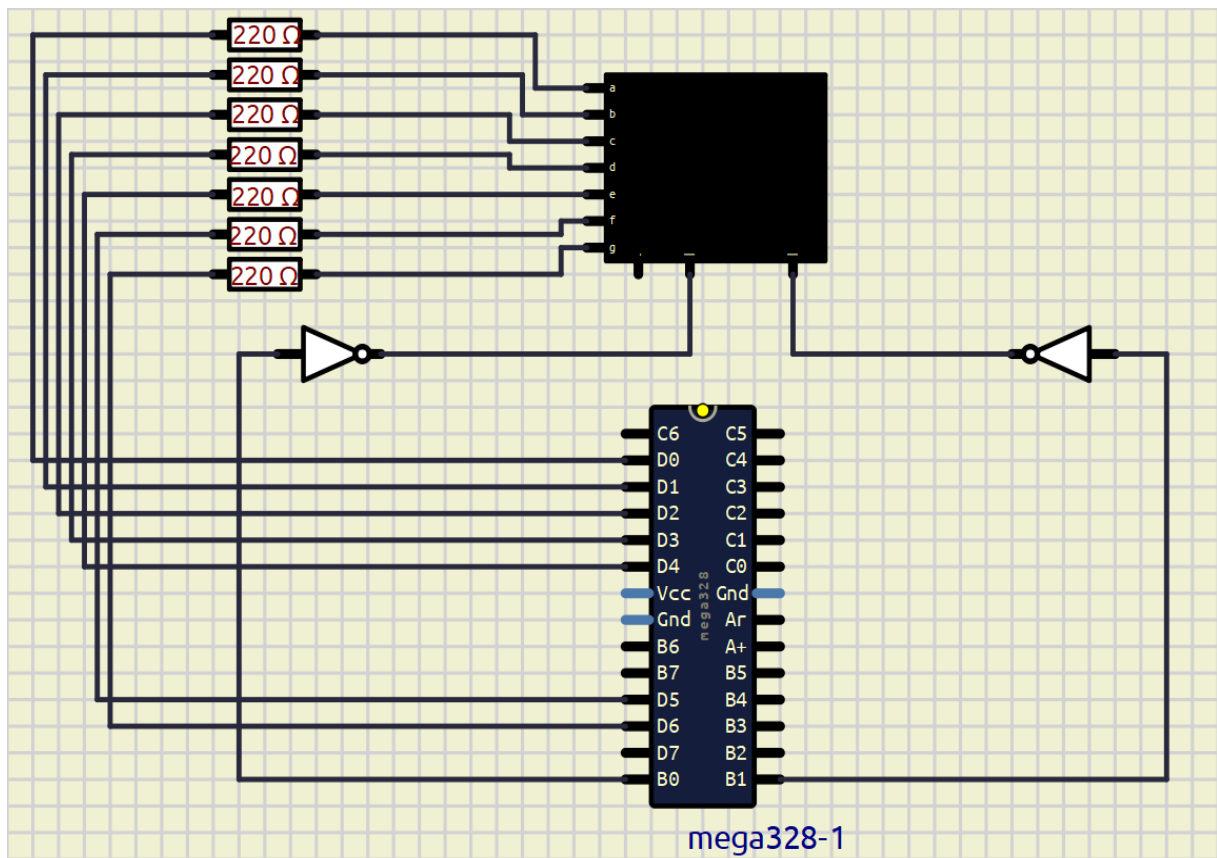
LOW). If the common pin were set to HIGH, no LED would glow as no current would flow in any of the LEDs. So, we conclude that we need the common pin to go LOW for that display to glow. However, this is a bit contradictory to how human understands this. Therefore, we use two logic inverters – NOT gates on the paths – B0 to common pin of left side and B1 to common pin of right side. This ensures our logic (and code) remains valid and the circuit works as intended.

The decimal pin on the 7-segment display is left unconnected as it is of no relevance here. An important element to include is the resistor, one of which is connected to each of the 7 segment pins (a to g). These 7 resistors ensure that the current never exceeds the safe limit and the display remains safe; this precaution is more relevant to real world application and won't affect the simulation much, other than a subtle decrease in the brightness of the LEDs.

An interesting question to ask is – how are we controlling a total of 14 segments plus 2 common cathodes only with 7 pins (D0-D6) and two additional PORTB pins? The answer is Multiplexing. Multiplexing interchanges the HIGH and LOW of B0 and B1 so fast (at each interval of 5 milliseconds) that due to the persistence of vision it appears as if both displays are lit up simultaneously, which in reality they are not. Hence the same PORTD can be used to control both the displays. This comes in handy particularly when we have a lot of displays and only a limited number of microcontrollers.

Another property of microcontroller worth discussing is its speed. Speed of a microcontroller determines how fast the chip's internal "brain" can think and act. It has a tiny component called oscillator (a quartz crystal). When we wrote `"#define F_CPU 16000000UL"` in our code, we defined exactly the frequency (16MHz) at which every single action – like adding two numbers or checking if a button is pressed – is executed. The "UL" simply stands for 'unsigned long'.

CIRCUIT DIAGRAM



CHALLENGES

- Couldn't find ATmega328p in the list of components under AVR in Simulide. Used ATmega328 instead. While there is a difference between the two, it won't matter here.
- Downloading Microchip studio was a total chaos. First received a blank terms and conditions page after opening the exe, googled the error, found nothing. Then switched off the internet and retried opening the exe and this time it worked. However, later it need internet again to download some components. Wasted half a day on installing the software.
- The coding part was the most laborious. I wanted to add the special effect of "thrice blink" (just a wish) when the timer reached 00 before resetting so I created a function for the same. It was fun and challenging.